

Software Requirements Specification (SRS)

Natural-Language Task Manager

Technology stack: MERN (MongoDB, Express.js, React, Node.js) + Google Gemini API **Version:** 1.0

1. Introduction

1.1 Purpose

This document describes the requirements for the Natural-Language Task Manager, a web application where users type tasks in plain English and AI converts them into structured tasks (title, due date, priority, labels).

1.2 Scope

Users normally fill many form fields to create a task. This system removes that effort. A user types a sentence such as *"submit report by Friday 5pm, high priority"*, and the system:

- extracts the title, due date/time, priority and labels using Gemini,
- shows a preview for the user to confirm or edit,
- saves the task and shows it in a list view and a calendar view,
- sends reminders before the due date.

The same system can be used for team tasks with assignees, student assignment tracking, or daily standup planning.

1.3 Definitions

Term	Meaning
SRS	Software Requirements Specification
NLP	Natural Language Processing
Gemini	Google's AI model used to parse text into structured data
JWT	JSON Web Token, used for login sessions
CRUD	Create, Read, Update, Delete

1.4 Intended Readers

Developers, project guide/professors, testers and evaluators.

2. Overall Description

2.1 Product Perspective

A standalone web application with a React frontend, a Node/Express REST API, a MongoDB database, and an

A standalone web application with a React frontend, a Node/Express REST API, a MongoDB database, and an external AI service (Gemini API).

2.2 Product Functions (Summary)

- 1. User registration and login
- 2. Natural-language task input
- 3. Parsed task preview before saving
- 4. Priorities and labels
- 5. List view and calendar view
- 6. Due-date reminders
- 7. Edit, complete and delete tasks

2.3 User Classes

User	Description
Registered User	Creates and manages own tasks
Admin (optional)	Manages users and views system data

2.4 Operating Environment

- Any modern browser (Chrome, Edge, Firefox, Safari)
- Server: Node.js 18+ ; Database: MongoDB (local or Atlas)

2.5 Constraints

- Needs internet access for the Gemini API
- Gemini API has usage limits and may be slow or unavailable at times
- API keys must be kept on the server, never in the frontend

2.6 Assumptions

- Users write tasks in English
- Users' time zone is known (from browser or profile)
- If no year or time is given, the system uses sensible defaults (current year, e.g. 9:00 AM)

3. System Features (Functional Requirements)

FR-1 User Authentication

- FR-1.1 The system shall allow users to register with name, email and password.
- FR-1.2 The system shall allow login with email and password.
- FR-1.3 Passwords shall be stored hashed (e.g. bcrypt).
- FR-1.4 The system shall use JWT to keep users logged in.
- FR-1.5 The system shall allow logout

- FR-1.3 The system shall allow logout.

FR-2 Natural-Language Task Input

- FR-2.1 The system shall provide a single text box for typing a task in plain English.
- FR-2.2 The system shall send the text and the current date/time/time zone to the backend.
- FR-2.3 The backend shall call Gemini to extract: title, description (optional), due date, due time, priority, labels.
- FR-2.4 Relative dates ("tomorrow", "next Monday", "in 3 days") shall be converted to exact dates.

FR-3 Parsed Task Preview

- FR-3.1 Before saving, the system shall show the parsed result in a preview card.
- FR-3.2 The user shall be able to edit any field in the preview.
- FR-3.3 The user shall be able to confirm (save) or cancel.
- FR-3.4 If parsing fails or is unclear, the system shall show an error and allow manual entry.

FR-4 Priorities and Labels

- FR-4.1 Priority levels: Low, Medium, High (default: Medium).
- FR-4.2 Users shall be able to add, change and remove labels (e.g. "college", "work").
- FR-4.3 Users shall be able to filter tasks by priority and label.

FR-5 List View

- FR-5.1 The system shall show all tasks in a list.
- FR-5.2 Tasks shall be sortable by due date and priority.
- FR-5.3 Users shall be able to search tasks by title.
- FR-5.4 Overdue tasks shall be highlighted.

FR-6 Calendar View

- FR-6.1 The system shall show tasks on a calendar (month and week).
- FR-6.2 Clicking a task on the calendar shall show its details.
- FR-6.3 Users shall be able to drag a task to another date to reschedule (optional).

FR-7 Due Reminders

- FR-7.1 The system shall remind users before a task is due (e.g. 1 hour and 1 day before).
- FR-7.2 Reminders shall be shown in-app and may also be sent by email.
- FR-7.3 Users shall be able to turn reminders on or off.

FR-8 Task Management

- FR-8.1 Users shall be able to edit, delete, and mark tasks as complete or incomplete.
- FR-8.2 Users shall only see and change their own tasks.

FR-9 Use-Case Extensions (choose as per project scope)

Option	Extra features
A) Team tasks	Assign task to a team member ("assign to Rahul"), team member list, assignee shown in task
B) Student assignment tracker	Subject/course label, assignment deadlines, progress status
C) Daily standup planner	Daily summary: done yesterday, plan today, blockers

4. External Interface Requirements

4.1 User Interface

- Clean, responsive design for desktop and mobile
- Main screens: Login/Register, Dashboard (input box + list), Calendar, Task details/edit
- Preview card shown after the user submits text

4.2 Software Interfaces

Interface	Purpose
Gemini API	Converts text to structured task JSON
MongoDB	Stores users and tasks
Email service (e.g. Nodemailer)	Sends reminder emails

4.3 Communication Interface

- REST API over HTTPS, data in JSON

4.4 Sample API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login
POST	/api/tasks/parse	Send text, get parsed preview
POST	/api/tasks	Save confirmed task
GET	/api/tasks	Get all tasks (with filters)
PUT	/api/tasks/:id	Update task
DELETE	/api/tasks/:id	Delete task

5. Data Requirements

5.1 User

`_id, name, email, passwordHash, timeZone, createdAt`

5.2 Task

`_id, userId, title, description, dueDate, priority (low/medium/high), labels[], status (pending/completed), reminderSent, originalText, createdAt, updatedAt`

(For Option A add assignee; for Option B add subject.)

5.3 Sample Parsed Output

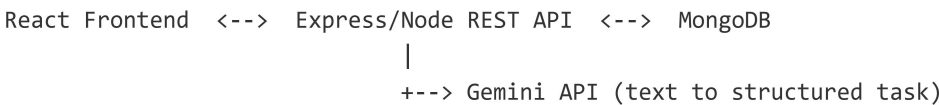
Input: *"submit report by Friday 5pm, high priority"*

```
{
  "title": "Submit report",
  "dueDate": "2026-10-09T17:00:00",
  "priority": "high",
  "labels": []
}
```

6. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	Parsing result shown within 5 seconds; pages load within 3 seconds
NFR-2	Security	Hashed passwords, JWT auth, API key stored on server, input validation
NFR-3	Reliability	If Gemini fails, user can still create tasks manually
NFR-4	Usability	Simple UI; a new user can add a task in under 1 minute
NFR-5	Compatibility	Works on latest versions of major browsers
NFR-6	Scalability	Should support many users with indexed database queries
NFR-7	Maintainability	Modular code (separate routes, controllers, models)
NFR-8	Privacy	Users' tasks are visible only to them (or their team in Option A)

7. System Architecture (Overview)



+++> Email service (reminders)

Flow: User types text → Frontend sends to `/api/tasks/parse` → Backend asks Gemini → Parsed task returned → User confirms in preview → Task saved in MongoDB → Shown in list/calendar → Reminder sent near due time.

8. Other Requirements

- **Error handling:** clear messages for invalid input, network errors and AI failures.
 - **Legal/ethical:** user data is not shared with third parties except the text sent to Gemini for parsing.
-

9. Future Enhancements

- Voice input for tasks
 - Recurring tasks ("every Monday")
 - Mobile app / push notifications
 - Google Calendar sync
 - Smart suggestions for priority based on past behavior
-

10. Conclusion

The Natural-Language Task Manager reduces the effort of task entry by using AI to understand plain English, while the preview step keeps the user in control of what gets saved.

